

10 SPARK DATAFRAME OPERATIONS EVERY DATA ENGINEER MUST KNOW

MASTER THESE OPERATIONS TO TRANSFORM, CLEAN, AGGREGATE, AND
ANALYZE DATA EFFICIENTLY IN APACHE SPARK.



BY MICHAEL FERNANDES

1. select()

Selects specific columns from a DataFrame.

Instead of reading all columns, select only the ones you need to improve performance.



```
df.select("customer_id", "name",  
"salary").show()
```

INPUT

customer_id	name	age	salary
101	John	25	50000
102	Doe	28	60000

OUTPUT

customer_id	name	salary
101	John	50000
102	Doe	60000

2. withColumn()

Creates a new column or modifies an existing one.
It is used in Feature engineering and business calculations.



```
from pyspark.sql.functions import col

df = df.withColumn(
    "annual_salary",
    col("salary") * 12
)

df.show()
```

OUTPUT

name	salary	annual_salary
John	50000	600000
Doe	60000	700000

3. filter()

Filters rows based on conditions.
It is used to reduce dataset size before expensive operations.

```
df.filter(col("salary") > 50000).show()
```

INPUT

customer_id	name	age	salary
101	John	25	50000
102	Doe	28	40000

OUTPUT

customer_id	name	salary
101	John	50000

4. drop()

Removes unwanted columns.
It is used to Keep datasets clean and reduces memory usage.



```
df.drop("age").show()
```

INPUT

customer_id	name	age	salary
101	John	25	50000
102	Doe	28	60000

OUTPUT

customer_id	name	salary
101	John	50000
102	Doe	60000

5. orderBy()

It Sorts rows based on conditions- it ranks data in ascending or descending order
It is used for Ranking and reporting.



```
df.orderBy(col("salary").desc())  
.show()
```

OUTPUT

customer_id	name	salary
101	John	80000
102	Doe	70000
103	Alice	60000
104	Bob	50000

6. groupBy()

It Groups rows by a column.
It is used before aggregation.



```
df.groupBy("department").count()  
.show()
```

INPUT

customer_id	name	department
101	John	IT
102	Doe	IT
103	Alice	HR

OUTPUT

department	count
IT	2
HR	1

7. agg()

It performs aggregate calculations.
Used to Summarizes data.



```
from pyspark.sql.functions import sum

df.groupBy("department")
  .agg(sum("salary").alias("total_salary"))
  .show()
```

OUTPUT

department	total_salary
IT	450000
HR	180000

8. join()

It Combines two DataFrames.
It is used in Real-world data lives in multiple tables.

CUSTOMERS

customer_id	name
101	John
102	Doe

ORDERS

customer_id	order_amt
101	1000
102	1500

```
customers.join(  
    orders,  
    on="customer_id",  
    how="inner"  
).show()
```

OUTPUT

customer_id	name	order_amt
101	John	1000
102	Doe	1500

9. distinct()

Distinct Removes duplicate rows.
It Improves data quality.

```
df.select("city").distinct().show()
```

INPUT

customer_id	name	age	city
101	John	25	Mumbai
102	Doe	28	Delhi
103	Alice	30	Mumbai

OUTPUT

city
Mumbai
Delhi

10. union()

Combines rows from two DataFrames.
It is used for Merging daily or monthly datasets.

January

city	sales
Mumbai	1000
Delhi	2000

ORDERS

city	sales
Mumbai	3000
Delhi	4000

```
from pyspark.sql.functions import sum

jan_df.union(order_df).groupBy("city")
    .agg(sum("sales").alias("total_sales")) \
    .show()
```

OUTPUT

city	total_sales
Mumbai	4000
Delhi	6000

SUMMARY

SELECT() → CHOOSE COLUMNS

WITHCOLUMN() → CREATE/MODIFY COLUMNS

FILTER() → FILTER RECORDS

DROP() → REMOVE COLUMNS

ORDERBY() → SORT DATA

GROUPBY() → GROUP RECORDS

AGG() → AGGREGATE DATA

JOIN() → COMBINE DATASETS

DISTINCT() → REMOVE DUPLICATES

UNION() → MERGE DATASETS

The background of the image consists of numerous thin, light gray wavy lines that flow from the left side towards the right, creating a sense of movement and depth. These lines are of varying frequencies and amplitudes, giving the background a textured, organic feel.

THANK YOU